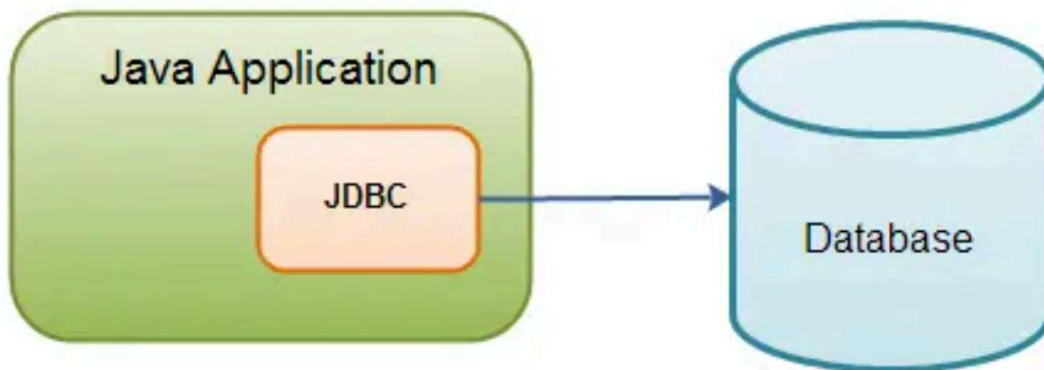


JDBC (Java Database Connectivity)



- JDBC is an API in Java used to connect Java applications with databases



What is an API?

- API = Application Programming Interface

Simple meaning:

API is a collection of Interfaces, Classes, Methods, and Rules that [allows](#) two software programs to communicate.

Example

Customer → Waiter(API) → Kitchen

- You don't [directly go to the kitchen](#). You talk to the waiter.

Similarly:

Java [Program](#) → JDBC [API](#) → Database.

JDBC acts [like](#) a waiter.

Why is JDBC Needed?

- Database vendors are different: MySQL, Oracle, PostgreSQL, SQL Server. Each database has a different internal implementation.
- Java needed ONE COMMON WAY to communicate with all databases.
- So Sun Microsystems created the JDBC API.

Why is Java-DB communication required?

- RDBMS are excellent in data storage in a secure and easily retrievable manner, but they are very poor in processing and presenting the data in a user-understandable format.
- Java is excellent at processing and presenting data, but Java is very poor at storing the data
(In Java, we can store the business data permanently using Serialization and files.)
- So, in order to develop a powerful business application, we need to communicate our Java Application with the DB server.

JDBC is NOT:

- SOFTWARE
- Database
- Driver
- Server

JDBC is a set of interfaces/classes inside Java.

So, JDBC is an API that helps applications to communicate with databases. It allows Java programs to connect to a database, run queries, retrieve, and manipulate data. Because of JDBC, Java applications can easily work with different relational databases like MySQL, Oracle, PostgreSQL, and more.

Interfaces of JDBC API :

Class/Interface	Description
<u>DriverManager</u>	This class manages a list of database drivers. Matches connection requests from the Java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain sub-protocol under JDBC will be used to establish a database Connection.
<u>Driver</u>	This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. In stead, you use <u>DriverManager</u> objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.
<u>Connection</u>	This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
<u>Statement</u>	You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
<u>ResultSet</u>	This class retrieves data from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
<u>SQLException</u>	This class handles any errors that occur in a database application.

JDBC steps

1. Load a JDBC driver.
2. Create [connections](#).
3. Connect [to](#) the data source.
4. Execute SQL statement(s).
5. Map the results to data structures.

Load a JDBC driver

JDBC Driver: Driver is the actual software/library that understands the database language.

For MySQL:

```
Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
```

For Oracle:

```
Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();
```

For MS SQL Server :

```
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver").newInstance();
```

To connect your Java application to a database, you first initialize the driver and register it with the **DriverManager**. This allows the application to communicate with your specific SQL or NoSQL database.

Common syntax: `Class.forName("com.mysql.cj.jdbc.Driver");`

What Does Class.forName() Do?

- Does [it](#) create an [object](#)?

- Not directly.
- It:
 - Loads **class** into JVM memory
 - Executes a **static** block

Internally, the Driver Class Looks Like

```
public class Driver
implements java.sql.Driver {
    static {
        DriverManager.registerDriver(
            new Driver()
        );
    }
}
```

So **When This Executes** `Class.forName("com.mysql.cj.jdbc.Driver");`
JVM:

1. Loads the **Driver class**
2. Static block runs automatically
3. **Driver** registers itself

THIS IS CALLED Loading + Registering Driver

Create Connections

DriverManager in JDBC: **DriverManager** is one of the MOST IMPORTANT classes in JDBC that manages JDBC drivers, establishes database connections, and selects the appropriate driver.

A Java application does NOT know MySQL internal communication, Oracle protocol, PostgreSQL protocol, and different databases have different Drivers, so DriverManager acts as a middle layer.

Syntax : `DriverManager.getConnection(url,user,password);`

Example :

```
Connection con =
    DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/test", "root", "root");
```

Create Statement

Statement is an **interface** used to send SQL *queries* to the **database**, *execute* SQL *statements*, and receive results.

Syntax:

```
Statement st = con.createStatement();  
con.createStatement();
```

executes internally: Connection **object** talks to DB session, and **Statement** object is created.

```
public class Demo {  
    public static void main(String[] args)  
        throws Exception {  
        // Load Driver  
        Class.forName("com.mysql.cj.jdbc.Driver");  
        // Establish Connection  
        Connection con = DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/test",  
            "root",  
            "root");  
  
        // Create Statement  
        Statement st =  
            con.createStatement();  
        System.out.println("Statement Created");  
        con.close();  
    }  
}
```

Statement Method: -

Method	Purpose
executeQuery()	SELECT
executeUpdate()	INSERT, UPDATE, DELETE

execute()	Any SQL
addBatch()	Batch processing

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class Demo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test",
                "root",
                "root" );
            Statement st = con.createStatement();

            // insert
            int a = st.executeUpdate( "insert into student values(1,'Rahul')");
            System.out.println("Inserted : " + a);

            // update
            int b = st.executeUpdate(update student set name='Vivek' where id=1"
);
            System.out.println("Updated : " + b);

            // delete
            int c = st.executeUpdate( "delete from student where id=1");
            System.out.println("Deleted : " + c);

            // inserting again for select
            st.executeUpdate( "insert into student values(1,'Aman')");
            st.executeUpdate("insert into student values(2,'Karan')");

```

```

// select using executeQuery
ResultSet rs = st.executeQuery("select * from student");
System.out.println("\nStudent Data");
while(rs.next()) {
    System.out.println(
        rs.getInt(1) + " " +
        rs.getString(2)
    );
}

// execute()
boolean check = st.execute("select * from student" );
if(check) {
    ResultSet r = st.getResultSet();
    System.out.println("\nData using execute()");
    while(r.next()) {
        System.out.println(
            r.getInt(1) + " " +
            r.getString(2)
        );
    }
}

// batch
st.addBatch("insert into student values(3,'Simran')");
st.addBatch("insert into student values(4,'Mohit')");
st.addBatch("update student set name='Akash' where id=2");
int arr[] = st.executeBatch();
System.out.println("\nBatch Executed");
for(int i : arr) {
    System.out.println(i);
}

// final table
ResultSet rs2 = st.executeQuery("select * from student");
System.out.println("\nFinal Data");
while(rs2.next()) {

```

```

        System.out.println(
            rs2.getInt(1) + " " +
            rs2.getString(2)
        );
    }
    con.close();
}
catch (Exception e) {
    System.out.println(e);
}
}
}

```

PreparedStatement in JDBC

PreparedStatement is a child interface of Statement used for dynamic queries, for better performance, and to prevent SQL Injection.

```
String q ="insert into student values("+ id + "," + name + ")";
```

Problem with Statement Query:- It compiled every time, **SQL Injection is possible, and not secure.**

SQL Injection: SQL injection (SQLi) is a critical web security vulnerability that allows attackers to interfere with database queries, often enabling them to view, modify, or delete sensitive data. **It occurs when untrusted user input is directly concatenated into SQL commands.**

Simple Meaning: Instead of giving normal input, the user gives SQL code as input, and the database executes it.

Example:-

```
select * from users where username='Rahul' and password='123';
```

Problematic Java Code

```
String username = "Vivek";
String password = "123";
```



```
String q = "select * from users where username='"+ username +"' and password='"+ password + "'";
```

Query Becomes:-

```
select * from users where username='Vivek' and password='123' ;  
(Works fine.)
```

Query with SQL injection:-

Suppose a hacker enters the password: '111' OR '1'='1' ;

So query becomes:-

```
select * from users where username='Rahul' and password='111' OR '1'='1' ;
```

Note:-

- '1'='1' is ALWAYS TRUE, so the database returns login success without a correct password. This is called an SQL Injection Attack.
- **But PreparedStatement prevents SQL Injection** because it uses placeholders (?) and treats user input as normal data instead of executable SQL code, so even if a hacker enters input like ' OR '1'='1', the database considers it a plain string and does not execute it as part of the SQL query.

In PreparedStatement, the query compiles only once, only values change, and PreparedStatement is Faster, more secure, and reusable.

Syntax:

```
PreparedStatement ps =con.prepareStatement("insert into student  
values(?,?)");
```

"?" Called:- Placeholder / Parameter Marker value will come later, means value will replace "?" later.

- In place of Placeholders, before executing the query, we need to bind the appropriate values to the corresponding placeholders by calling various types of "**setXXX(--)**" methods on the PreparedStatement object.
- Here, XXX will be replaced with the appropriate data type of the value.

Example:-

```
For Integer:- ps.setInt(1, 50);  
For String:- ps.setString(2,"Amit");  
and so on..
```

- After binding the appropriate placeholders, we need to execute the query by using one of the following methods of the PreparedStatement object.

```
public int executeUpdate(); // if dml
public ResultSet executeQuery() // if dql
public boolean execute(); // any type of sql
```

Example:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
public class Demo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test",
                "root",
                "root"
            );
            String q = "insert into student values(?,?)";
            PreparedStatement ps = con.prepareStatement(q);
            ps.setInt(1, 101);
            ps.setString(2, "Rahul");
            int rows = ps.executeUpdate();
            System.out.println(rows + " row inserted");
            ps.close();
            con.close();
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

ResultSet

ResultSet is an object that stores rows returned from the database from a SELECT query.

Suppose database table:

id	name
1	Aman
2	Rahul

After SELECT query:

```
ResultSet rs = st.executeQuery("select * from student");  
that table data comes into: rs
```

next() Method:

Initially, the cursor is before the first row, NOT on the first row, and next() Method Moves cursor forward **rs.next()**. next() returns a boolean. Return **true** Meaning row exists and **false** if no more rows exist.

```
while(rs.next()) {  
    System.out.println(  
        rs.getInt(1) + " " +  
        rs.getString(2)  
    );  
}
```

Complete Example

```
import java.sql.*;  
public class Demo {  
    public static void main(String[] args)  
        throws Exception {  
        Class.forName("com.mysql.cj.jdbc.Driver");  
        Connection con = DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/test",  
            "root",  
            "root");  
        Statement st = con.createStatement();  
        ResultSet rs = st.executeQuery("select * from student");  
        while(rs.next()) {  
            System.out.println(  
                rs.getInt(1) + " " +
```

```

        rs.getString(2));
    }
    con.close();
}
}

```

Data Fetching Methods of Result Set

Method	Purpose
getInt()	int value
getString()	String value
getDouble()	double value
getBoolean()	boolean value
getDate()	Date value

Summary

JDBC is the bridge between Java applications and databases. It provides a standard API through which Java can communicate with different databases like MySQL, Oracle, PostgreSQL, etc.

In JDBC, we learned:

- Loading and registering the driver
- Establishing connection
- DriverManager
- Statement
- executeQuery()
- executeUpdate()
- execute()
- ResultSet and cursor
- CRUD operations
- Internal working of JDBC

ResultSetMetaData

- ResultSetMetaData is an interface in JDBC used to get information about the columns of a ResultSet object.

Meaning of Metadata

Metadata Means Data about data

Example:-

Actual Data

Id	name	marks
101	Rahul	85

Metadata

- Column Names
- Column Count
- Column Types
- Table Name

Package

java.sql.ResultSetMetaData

Why Do We Use ResultSetMetaData?

We use it to get:

- total columns
- column names
- column data types
- table name

How to Create ResultSetMetaData Object?

Step 1 → Execute Query

```
ResultSet rs = st.executeQuery("select * from student");
```

Step 2 → Get Metadata Object

```
ResultSetMetaData rsmd = rs.getMetaData();
```

Important Methods of ResultSetMetaData

Method	Purpose
getColumnCount()	Returns total columns
columnName(int column)	Returns column name
getColumnTypeName(int column)	Returns data type
getTableName(int column)	Returns table name
isNullable(int column)	Checks if null is allowed or not

Example

```
public class Demo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.out.println(e);
        }
        String url = "jdbc:mysql://localhost:3306/Chitkara";
        try(Connection con = DriverManager.getConnection(url,"root","root")){
            PreparedStatement ps = con.prepareStatement("select * from Student");
            ResultSet rs = ps.executeQuery();
            ResultSetMetaData rsmd = rs.getMetaData();

            int numberOfColumn = rsmd.getColumnCount();
            System.out.println("Total column numbers :-"+numberOfColumn);

            System.out.println("=====");

            for(int i=1; i<=numberOfColumn; i++) {
                System.out.println(rsmd.getColumnName(i));
            }
            System.out.println("=====");
            System.out.println(rsmd.getTableName(1));
        } catch (Exception e) {

        }
    }
}
```

Output:-

Total column numbers :-3

=====

roll

name

marks

=====

student

Database:-

+-----	+-----	+-----+
roll	name	marks
+-----	+-----	+-----+
1	Abhishek	10
6	Sarthak	110
+-----	+-----	+-----+

When the query executes:

- database sends row data
- The database also sends column information
- The ResultSetMetaData object is used to access the column information.

Dynamic Data Printing Example

```
public class Demo {  
    public static void main(String[] args) {  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");  
        } catch (ClassNotFoundException e) {  
            System.out.println(e);  
        }  
        String url = "jdbc:mysql://localhost:3306/Chitkara";  
        try(Connection con = DriverManager.getConnection(url,"root","root")){  
            PreparedStatement ps = con.prepareStatement("select * from Student");  
            ResultSet rs = ps.executeQuery();  
            ResultSetMetaData rsmd = rs.getMetaData();  
  
            int numberOfColumn = rsmd.getColumnCount();  
  
            while(rs.next()) {
```

```

        for(int i=1; i<=numberOfColumn; i++) {
            System.out.print(rs.getString(i)+" ");
        }
        System.out.println();
    }

    }catch(Exception e) {
        System.out.println(e);
    }
}
}

```

Output:-

1 Abhishek 10

6 Sarthak 110

Mini Project's

Bank Management System

Features:

- Add Account
- Deposit
- Withdraw
- Transfer Money
- Mini Statement

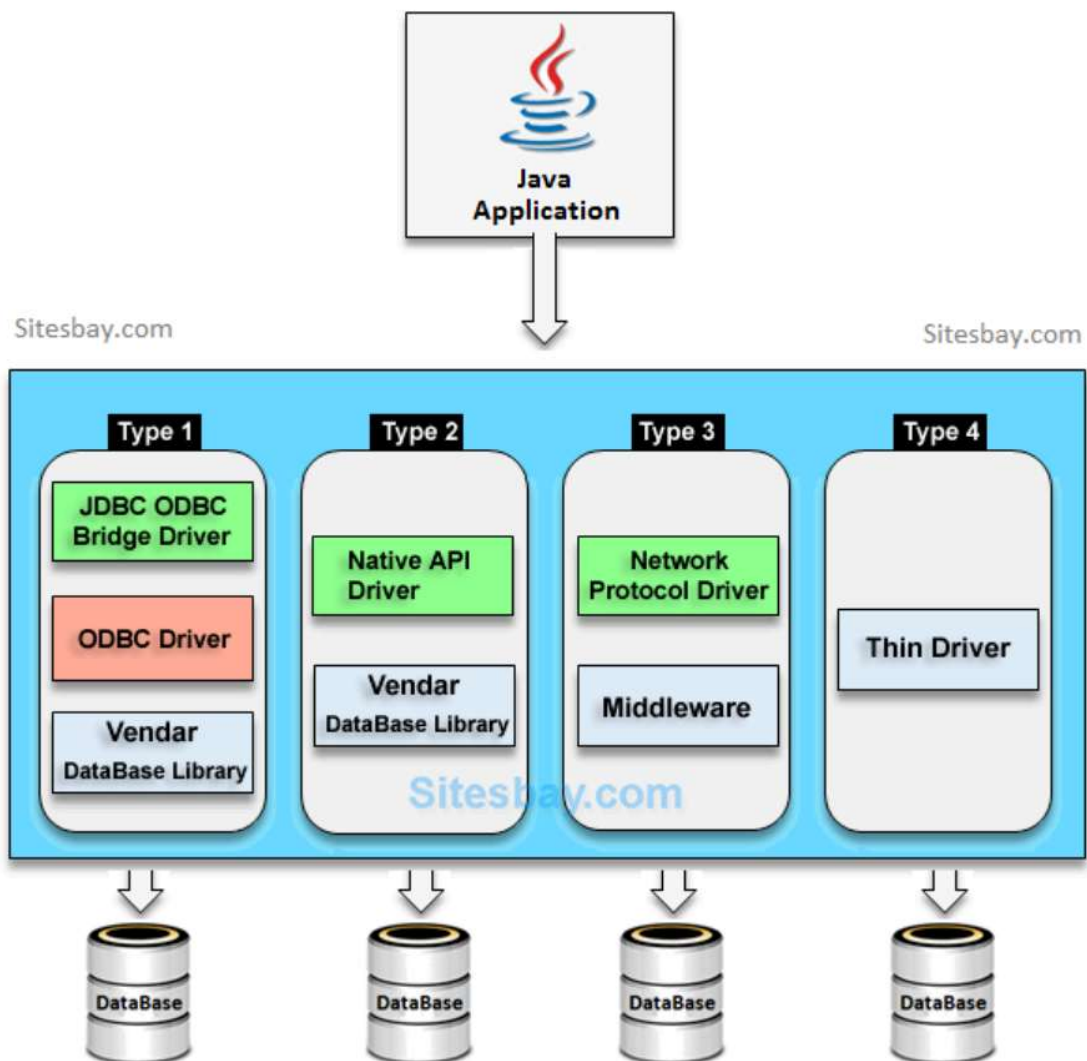
Employee Management System

Operations:

- add employee
- update salary
- search employee

- delete employee

JDBC Drivers



- JDBC drivers are software components that help Java applications communicate with databases.

There are 4 Types of JDBC Drivers :

1. Type 1 Driver: JDBC-ODBC Bridge Driver

- ODBC (Open Database Connectivity):
- ODBC is an API developed by Microsoft, used for connecting applications with databases.
- Earlier, every database had a different communication system, so Microsoft created a common standard called ODBC

Example:

- Oracle different
- MySQL different
- SQL Server different

So, ODBC acts like a universal translator between the application and the database

JDBC-ODBC Bridge:

Java initially could not directly talk to databases, so Sun Microsystems created the JDBC-ODBC Bridge Driver, which means JDBC calls are converted into ODBC calls, then ODBC communicates with the DB.

Problem with Type 1 Driver:

It is Platform Dependent and ODBC, mainly Windows-based, with ODBC Installation Needed in every machine, so it was removed in Java 8 because it was outdated, slow, and platform dependent. Type 4 drivers became better.

2. Type 2 Driver: Native API Driver

- Type 2 driver is the second type of JDBC driver. It uses Native Database Libraries written in C/C++. Type 2 was introduced because Type 1 was too slow and had too many layers, so Type 2 removed the ODBC bridge and directly used native database APIs.

Native Library:

Database vendors provide libraries written in C / C++ example, .dll files in Windows, .so files in Linux. These libraries directly communicate with the database, which means Java does not directly talk to the database. Java calls native code, and native code talks to the database.

Problem with Type 2 Driver:

It is platform-dependent because it uses OS-specific native libraries (.dll/.so files), and native library installation is required on every machine, making deployment and maintenance difficult, along with possible security risks, so Type 4 drivers became more preferred.

3. Type 3 Driver: Network Protocol Driver

- Type 3 driver is the third type of JDBC driver. It uses a Middleware Server between the Java application and the database. Type 3 was introduced to solve the problems of Type 1 and Type 2 drivers, like platform dependency and native library installation.

Middleware Server:

The middleware server acts as a middle communication layer between the Java application and the database. A Java application sends JDBC calls to the middleware server, then the middleware converts those calls into a database-specific protocol and communicates with the database. This means Java does not directly talk to the database.

Problem with Type 3 Driver:

It requires a separate middleware server for communication, which increases network overhead, complexity, and maintenance. Due to the extra middle layer, performance becomes slower, so Type 4 drivers became more preferred because they directly communicate with the database.

4. Type 4 Driver: Thin Driver / Pure Java Driver

- Type 4 driver is the fourth and most used type of JDBC driver. It is called Thin Driver because it directly communicates with the database without using ODBC, native libraries, or a middleware server. It is completely written in Java.

Pure Java Driver:

Type 4 driver converts JDBC calls directly into database-specific protocol and communicates with the database using a socket connection. This means a Java application directly talks to the database without any middle layer.

Example :

- MySQL Driver → `com.mysql.cj.jdbc.Driver`
- Oracle Driver → `oracle.jdbc.driver.OracleDriver`

Advantages of Type 4 Driver :

- Fastest JDBC driver

- Platform independent
- No native library required
- No middleware server required
- Easy to deploy and maintain
- Most preferred in real projects

Problem with Type 4 Driver:

It is database dependent because different databases require different Type 4 drivers, but it became an industry standard because of high performance, simplicity, portability, and direct communication with the database.

Short Description of JDBC Drivers

- **Type 1 drivers** are database-independent and platform-dependent.
- **Type 2 drivers** are database-dependent and platform-dependent.
- **Type 3 drivers** are database-independent and platform-independent.
- **Type 4 drivers** are database-dependent and platform-independent.